

Encoding in Machine Learning

A Complete Guide — Types, Use Cases & Examples

Machine learning models are fundamentally mathematical. They cannot process raw text like 'Male/Female' or 'Red/Blue/Green' — every input must be numeric. Encoding is the process of converting categorical data into numbers that a model can learn from. Choosing the wrong encoding can mislead your model entirely.

Quick Reference

Type	What it does	Best for
Label	Dog=0, Cat=1, Bird=2	Target column / ordinal data
Ordinal	Low=0, Medium=1, High=2	Features with real order
One-Hot	Splits into binary columns	Nominal, few categories
Binary	Compresses via binary digits	Nominal, many categories
Target/Mean	Replaces with mean outcome	High cardinality features

1. Label Encoding

Label Encoding assigns a unique integer to each category. It is the simplest form of encoding and works well for the target/output variable or genuinely ordered data.

How it works:

Original Value	Encoded Value
Dog	0
Cat	1
Bird	2

Python Code:

```
from sklearn.preprocessing import LabelEncoder le = LabelEncoder() df['animal']  
= le.fit_transform(df['animal'])
```

When to use:

- ✓ Target/output column (what you are predicting)
- ✓ Ordinal features with a natural order (Small, Medium, Large)
- ✗ Nominal input features — the model falsely assumes Bird(2) > Cat(1) > Dog(0)

■ Warning: Applying Label Encoding to a nominal input feature introduces false ordinal relationships. Use One-Hot instead.

2. Ordinal Encoding

Ordinal Encoding is like Label Encoding, but you explicitly define the order. This prevents the encoder from assigning numbers randomly.

Example — Education Level:

Category	Encoded
High School	0
Bachelor's	1
Master's	2
PhD	3

Python Code:

```
from sklearn.preprocessing import OrdinalEncoder enc =  
OrdinalEncoder(categories=[['High School', 'Bachelor\'s', 'Master\'s', 'PhD']])  
df[['education']] = enc.fit_transform(df[['education']])
```

When to use:

- ✓ Feature has a meaningful, confirmed order
- ✓ You want to define the order explicitly (don't trust automatic assignment)
- ✗ Nominal features with no real order

3. One-Hot Encoding

One-Hot Encoding creates a new binary column for each category. Exactly one column is 1 (hot) for each row, the rest are 0. This is the safest default for nominal features.

Example — Color column:

Original	color_Red	color_Blue	color_Green
Red	1	0	0
Blue	0	1	0
Green	0	0	1

Python Code:

```
# Using pandas df = pd.get_dummies(df, columns=['color']) # Using sklearn from
sklearn.preprocessing import OneHotEncoder enc =
OneHotEncoder(sparse_output=False) encoded = enc.fit_transform(df[['color']])
```

When to use:

- ✓ Nominal features (no natural order)
- ✓ Low cardinality — few unique values (Gender, Season, Department)
- ✗ High cardinality — too many values (City, Product ID) → column explosion

■ **Dummy Variable Trap:** With k categories, One-Hot creates k columns. Drop one column (`drop_first=True` in pandas) to avoid multicollinearity in linear models.

4. Binary Encoding

Binary Encoding converts each category to an integer, then to its binary representation, then splits the binary digits into separate columns. It is a compression technique — far fewer columns than One-Hot for high cardinality data.

Step-by-step for City column:

City	Integer	Binary	Col 1	Col 2	Col 3
Delhi	1	001	0	0	1
Mumbai	2	010	0	1	0
Dhaka	3	011	0	1	1
Karachi	4	100	1	0	0

Colombo	5	101	1	0	1
---------	---	-----	---	---	---

5 cities → One-Hot needs 5 columns. Binary needs only 3. With 1000 cities: One-Hot = 1000 columns, Binary = 10 columns.

Python Code:

```
import category_encoders as ce  
enc = ce.BinaryEncoder(cols=['city'])  
df = enc.fit_transform(df)
```

When to use:

- ✓ High cardinality nominal features
- ✓ When One-Hot would create too many columns
- ✗ Low cardinality — One-Hot is simpler and equally effective

5. Target / Mean Encoding

Target Encoding replaces each category with the mean of the target variable for that category. Unlike other encodings, the number it assigns actually carries predictive meaning — it reflects how much that category relates to the outcome.

The Core Idea:

Instead of asking 'What number should I assign to this city?' it asks: 'What does this city tell us about the outcome?'

Example — Predicting Customer Purchase (1=bought, 0=didn't):

Customer	City	Bought?
1	Dhaka	1
2	Dhaka	1
3	Dhaka	0
4	Chittagong	0
5	Chittagong	0
6	Sylhet	1

Calculate group means:

City	Purchases	Mean (Encoded Value)
Dhaka	2 out of 3 bought	0.67
Chittagong	0 out of 2 bought	0.00
Sylhet	1 out of 1 bought	1.00

Result after encoding:

Customer	City (encoded)	Bought?
1	0.67	1
2	0.67	1
3	0.67	0
4	0.00	0
5	0.00	0

6	1.00	1
---	------	---

The model now knows: Dhaka customers buy 67% of the time, Chittagong almost never, Sylhet always. That is far more informative than Dhaka=0, Chittagong=1, Sylhet=2.

Python Code:

```
import category_encoders as ce
enc = ce.TargetEncoder(cols=['city'])
df['city'] = enc.fit_transform(df['city'], df['bought'])
```

The Leakage Problem:

When encoding Customer 1 (Dhaka), you included their own purchase in the 0.67 mean. The model sees 0.67 → bought=1 — a circular pattern. This inflates training performance but fails on real data.

■ Fix: Calculate the mean from all other rows except the current one, or strictly apply encoding inside each cross-validation fold — never on the full dataset at once. Most libraries handle this with a smoothing parameter.

When to use:

- ✓ High cardinality nominal features (City, Product ID, Zip Code)
- ✓ Tree-based models (Random Forest, XGBoost, LightGBM)
- ✓ When the category has a clear relationship with the target
- ✗ Linear models — use One-Hot or Binary instead
- ✗ Without careful leakage prevention

When to Use Which Encoding

Situation	Recommended Encoding
Column is the target/output variable	Label Encoding
Feature has a confirmed natural order	Ordinal Encoding
Nominal feature, few unique values (<15)	One-Hot Encoding
Nominal feature, many unique values (>15)	Binary or Target Encoding
Tree-based model + high cardinality	Target Encoding
Linear model + high cardinality	Binary Encoding
Unsure / default choice	One-Hot Encoding

One-Hot is the safe default. Only deviate when cardinality is high (too many columns) or you have a confirmed ordering in the feature.

Common Mistakes

Using Label Encoding on nominal input features

The model treats the integers as ordered. Dog < Cat < Bird becomes a false assumption that harms model performance. Use One-Hot for nominal inputs.

Applying Target Encoding on the full dataset

Causes data leakage — the encoding uses the target to create a feature, letting the model indirectly see the answer during training. Always encode inside CV folds.

One-Hot on high cardinality columns

A column with 500 unique cities creates 500 new columns. This bloats memory, slows training, and often hurts performance. Use Binary or Target Encoding.

Forgetting to encode at inference time

Fit the encoder on training data only. Apply (transform only, no fit) on validation and test data. Fitting again on test data causes leakage.

Ignoring unseen categories

A new city that wasn't in training will break your encoder at inference. Handle with a fallback (global mean for Target, 'unknown' bin for Binary).

Summary

Encoding	Columns Created	Preserves Order?	Handles High Cardinality
Label	1	Yes (artificial)	Yes
Ordinal	1	Yes (defined)	Yes
One-Hot	k	No	No
Binary	$\log_2(k)$	No	Yes
Target	1	No	Yes

k = number of unique categories in the column.